

Reporte del Problema 1: Predicción de compra de casa usando una red neuronal artificial

1. Introducción

El objetivo de este proyecto fue entrenar una red neuronal artificial capaz de predecir si un cliente comprará una casa o no, utilizando un conjunto de datos tomado de Kaggle llamado **Global House Purchase Decision Dataset**. Este problema corresponde a una tarea de **clasificación binaria**, ya que la variable objetivo solo puede tomar dos posibles valores: el cliente compra la casa o el cliente no compra la casa.

La variable que se desea predecir es decision, donde el valor 1 representa que el cliente sí compró la casa, mientras que el valor 0 representa que el cliente no realizó la compra.

Para resolver este problema se utilizó una red neuronal densa, también conocida como **Multilayer Perceptron** o **MLP**, ya que el dataset contiene información tabular, es decir, columnas numéricas y categóricas relacionadas con características de la propiedad, condiciones económicas del cliente y datos generales del contexto de compra.

2. Descripción general del dataset

El archivo utilizado fue global_house_purchase_dataset.csv. Este dataset contiene información relacionada con posibles compradores de casas y con las características de las propiedades disponibles. Entre las variables incluidas se encuentran datos como el país, la ciudad, el tipo de propiedad, el precio de la propiedad, el ingreso anual del cliente, el monto del préstamo, la tasa de interés, la duración del préstamo, el puntaje de crédito, el tamaño de la familia y otras variables relevantes para estimar si una persona decide comprar una casa.

El dataset incluye una columna llamada property_id, la cual funciona únicamente como identificador del registro o propiedad. Esta columna no se usó para entrenar el modelo, ya que no aporta información real para la predicción y podría introducir ruido en el aprendizaje de la red neuronal.

La columna objetivo fue decision, que indica la decisión final de compra.

3. Análisis inicial de los datos

Primero se cargó el archivo CSV utilizando la librería pandas. Después se realizó una exploración inicial para conocer la estructura del dataset. En esta etapa se

revisaron las dimensiones del conjunto de datos, los nombres de las columnas, los tipos de datos y la existencia de valores nulos.

También se analizó la distribución de la variable objetivo decision para identificar si las clases estaban balanceadas o desbalanceadas. Este paso fue importante porque, si existe una gran diferencia entre la cantidad de clientes que compran y los que no compran, el modelo podría aprender a favorecer solamente a la clase mayoritaria.

Durante esta revisión se identificaron variables numéricas y variables categóricas. Las variables numéricas pueden ser procesadas directamente después de una normalización, mientras que las variables categóricas requieren ser transformadas a formato numérico antes de ingresar a la red neuronal.

4. Preprocesamiento de los datos

Antes de entrenar la red neuronal, fue necesario transformar los datos para que fueran adecuados para el modelo.

Primero se separó el dataset en dos partes: las variables predictoras X y la variable objetivo y. La variable objetivo fue la columna decision, mientras que las demás columnas fueron utilizadas como características de entrada, excepto property_id, que fue eliminada por ser un identificador.

Después se identificaron las columnas numéricas y categóricas. A las columnas numéricas se les aplicó StandardScaler, con el objetivo de normalizar sus valores. Esto es importante porque las redes neuronales suelen entrenar mejor cuando las variables numéricas tienen escalas similares. Por ejemplo, una variable como el precio de una casa puede tener valores muy grandes, mientras que una variable como la duración del préstamo puede tener valores mucho más pequeños. Si no se normalizan, las variables de mayor escala podrían dominar el entrenamiento.

Para las variables categóricas se utilizó OneHotEncoder. Esta técnica transforma categorías como país, ciudad o tipo de propiedad en columnas numéricas binarias. De esta forma, la red neuronal puede interpretar correctamente las variables categóricas.

El preprocesamiento se hizo utilizando ColumnTransformer, lo cual permitió aplicar diferentes transformaciones a columnas numéricas y categóricas dentro del mismo flujo de trabajo.

5. División del dataset

El conjunto de datos se dividió en tres subconjuntos:

- Conjunto de entrenamiento: 70% de los datos.

- Conjunto de validación: 15% de los datos.
- Conjunto de prueba: 15% de los datos.

La división se realizó de forma estratificada, es decir, procurando conservar la misma proporción de clases en los conjuntos de entrenamiento, validación y prueba. Esto fue importante porque el problema es de clasificación binaria y se quería evitar que alguno de los conjuntos quedara con una distribución muy diferente de la variable decision.

El conjunto de entrenamiento se utilizó para ajustar los pesos de la red neuronal. El conjunto de validación se utilizó para monitorear el desempeño del modelo durante el entrenamiento y seleccionar el mejor modelo. Finalmente, el conjunto de prueba se utilizó para evaluar el rendimiento final del modelo con datos que no fueron utilizados durante el entrenamiento.

6. Balanceo de clases

Al analizar la variable objetivo, se observó que las clases podían estar desbalanceadas, es decir, que podía existir una mayor cantidad de registros de una clase que de la otra. Para reducir el impacto de este problema, se utilizó `class_weight`.

El uso de `class_weight` permite asignar mayor peso a la clase minoritaria durante el entrenamiento. De esta manera, la red neuronal no se enfoca únicamente en predecir la clase más frecuente, sino que también aprende a reconocer los casos menos representados.

Esta estrategia ayuda a mejorar la capacidad del modelo para distinguir entre clientes que compran y clientes que no compran.

7. Arquitectura de la red neuronal

Para este problema se eligió una red neuronal densa tipo MLP, ya que el dataset es tabular. No se utilizaron redes convolucionales porque esas arquitecturas son más adecuadas para imágenes, donde existe una estructura espacial entre píxeles. En este caso, las variables son columnas independientes o características tabulares, por lo que una red densa es una opción más adecuada.

La arquitectura general utilizada fue la siguiente:

1. Una capa de entrada con tantas neuronas como variables resultaron después del preprocesamiento.
2. Varias capas ocultas densas con función de activación ReLU.

3. Capas Dropout para reducir el sobreajuste.
4. Una capa de salida con una sola neurona y función de activación sigmoid.

La función sigmoid se utilizó porque el problema es binario. Esta función produce un valor entre 0 y 1, que puede interpretarse como la probabilidad de que el cliente compre una casa.

La función de pérdida utilizada fue `binary_crossentropy`, que es apropiada para problemas de clasificación binaria. Como optimizador se utilizó Adam, ya que es un algoritmo eficiente y ampliamente usado para entrenar redes neuronales.

8. Entrenamientos realizados

Para cumplir con el objetivo de comparar varios entrenamientos, se entrenaron tres modelos diferentes con distintas configuraciones de capas ocultas, dropout y tasa de aprendizaje.

Los modelos entrenados fueron:

Modelo 1: MLP pequeña

Este modelo tuvo una arquitectura más simple, con dos capas ocultas. La intención de este entrenamiento fue tener una red base que sirviera como punto de comparación. Al ser una red más pequeña, tiene menor riesgo de sobreajuste, pero también puede tener menor capacidad para aprender relaciones complejas.

Modelo 2: MLP mediana

Este modelo utilizó una arquitectura más profunda, con tres capas ocultas. El objetivo fue permitir que la red aprendiera relaciones más complejas entre las variables de entrada y la decisión de compra. También se utilizó Dropout para evitar que el modelo memorizara demasiado los datos de entrenamiento.

Modelo 3: MLP con Batch Normalization

En este tercer modelo se agregó BatchNormalization. Esta técnica ayuda a estabilizar el entrenamiento de la red neuronal y puede mejorar la convergencia. También se utilizó Dropout, con el objetivo de reducir el sobreajuste.

Durante los entrenamientos se monitorearon las métricas de loss, accuracy y AUC, tanto en entrenamiento como en validación.

9. Métricas utilizadas

Se utilizaron varias métricas para evaluar el desempeño de los modelos.

La primera métrica fue accuracy, que mide el porcentaje de predicciones correctas. Sin embargo, debido a que el dataset puede estar desbalanceado, la exactitud por sí sola no siempre es suficiente para evaluar el modelo.

También se utilizó AUC, que mide la capacidad del modelo para separar correctamente las dos clases. Esta métrica es útil en problemas de clasificación binaria, especialmente cuando las clases no están perfectamente balanceadas.

Además, se generó una matriz de confusión para observar cuántos casos fueron clasificados correctamente e incorrectamente. La matriz de confusión permite analizar los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

Finalmente, se generó un reporte de clasificación con métricas como precision, recall y f1-score.

10. Selección del mejor modelo

El mejor modelo se seleccionó usando el valor de `val_auc`, es decir, el AUC obtenido en el conjunto de validación. Se eligió esta métrica porque permite evaluar mejor la capacidad del modelo para distinguir entre clientes que compran y clientes que no compran.

Una vez seleccionado el mejor modelo, se evaluó sobre el conjunto de prueba. Este paso fue importante porque el conjunto de prueba no fue utilizado durante el entrenamiento ni durante la selección del modelo, por lo que representa una evaluación más realista del desempeño final.

El mejor modelo obtenido fue: [colocar aquí el nombre del mejor modelo].

El AUC obtenido en el conjunto de prueba fue: [colocar aquí el AUC test].

La accuracy obtenida en el conjunto de prueba fue: [colocar aquí la accuracy test].

11. Gráficas generadas

Para cada entrenamiento se generaron gráficas de loss y accuracy.

Las gráficas de loss muestran cómo disminuye el error del modelo durante las épocas de entrenamiento. Se comparó la pérdida del conjunto de entrenamiento contra la pérdida del conjunto de validación. Esto permitió observar si el modelo estaba aprendiendo correctamente o si empezaba a sobreajustarse.

Las gráficas de accuracy muestran la exactitud del modelo durante el entrenamiento y la validación. Estas gráficas ayudaron a comparar el desempeño de los modelos y observar si el modelo mejoraba conforme avanzaban las épocas.

Además, se generó una matriz de confusión para el mejor modelo, con el fin de observar el comportamiento final del clasificador sobre el conjunto de prueba.

12. Relación con los notebooks de referencia

Para construir la solución se revisaron los notebooks proporcionados como referencia.

El notebook de Keras_Functional_API fue útil porque muestra cómo construir modelos usando la API funcional de Keras, definiendo explícitamente una capa de entrada, capas intermedias y una capa de salida. Esta estructura se adaptó para el problema de clasificación binaria con datos tabulares.

El notebook de Analisis_de_Sentimiento también fue relevante porque trabaja con clasificación binaria. De ahí se retomó la idea de usar una salida con activación sigmoid, la función de pérdida binary_crossentropy y el monitoreo de métricas durante el entrenamiento.

Los notebooks relacionados con Conv2D, redes residuales, aumento de datos y fine-tuning fueron analizados, pero no se usaron directamente porque están orientados principalmente a problemas de visión por computadora. En este proyecto no se trabajó con imágenes, sino con datos tabulares, por lo que una red neuronal densa fue una mejor elección.

13. Resultados generales

Después de entrenar los modelos, se compararon sus resultados en el conjunto de validación. El modelo con mejor desempeño fue elegido con base en la métrica val_auc.

Los resultados mostraron que la red neuronal pudo aprender patrones relevantes a partir de las características del cliente y de la propiedad. Variables como el precio de la casa, el ingreso anual, el monto del préstamo, la tasa de interés, el puntaje de crédito y otras características pueden influir en la decisión de compra.

El modelo final permite estimar la probabilidad de que un cliente compre una casa. Si la probabilidad generada por la red neuronal es mayor o igual a 0.50, el modelo clasifica el caso como compra. Si la probabilidad es menor a 0.50, lo clasifica como no compra.

14. Conclusión

En este proyecto se entrenó una red neuronal artificial para resolver un problema de clasificación binaria: predecir si un cliente comprará una casa o no. Para ello se utilizó un dataset tabular de Kaggle y se aplicó un proceso completo de preparación de datos, entrenamiento, validación y evaluación.

La estrategia seguida consistió en limpiar y preparar los datos, transformar variables categóricas con OneHotEncoder, normalizar variables numéricas con StandardScaler, dividir el dataset en entrenamiento, validación y prueba, y entrenar diferentes arquitecturas de redes neuronales densas.

Se eligió una arquitectura MLP porque el problema trabaja con datos tabulares y no con imágenes. También se utilizaron técnicas como Dropout, class_weight, validación y selección del mejor modelo por AUC.

Finalmente, se generaron gráficas de entrenamiento y validación para analizar el comportamiento de los modelos. Estas gráficas permitieron observar la evolución de la pérdida y la exactitud durante el entrenamiento. El modelo final fue evaluado con datos de prueba, obteniendo métricas que permiten medir su capacidad de generalización.

En conclusión, la red neuronal construida representa una solución adecuada para abordar el problema de predicción de compra de casa, ya que permite transformar variables económicas y de propiedad en una probabilidad de decisión de compra.